



SAPIENZA
UNIVERSITÀ DI ROMA

Architettura degli Elaboratori

UNIVERSITÀ DEGLI STUDI DI ROMA "LA SAPIENZA"

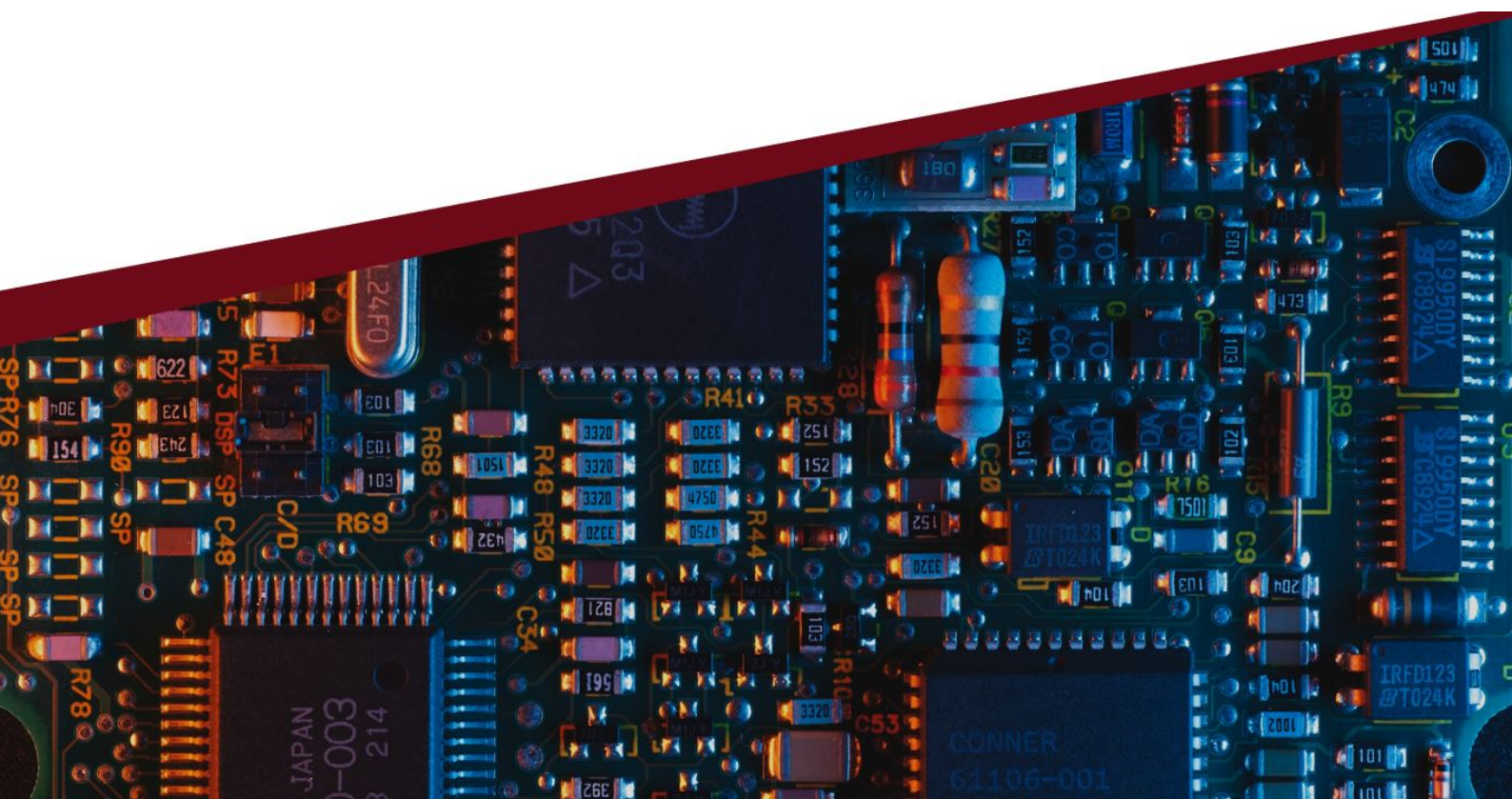
FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E
STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
EDOARDO PERSICHETTI

19 ottobre 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Indice

Prefazione	1
Introduzione al corso	5
I Introduzione alla CPU	9
1 Architettura di Von Neumann	11
2 Assembler MIPS	13
2.1 Architetture CISC e RISC	13
2.2 MIPS	13
2.3 Rappresentazione dell'istruzione	13
2.4 Tipi di indirizzamento alla memoria	15
II Progettare la CPU	19
3 Elaborare un'istruzione	21
4 Unità della CPU	23
4.1 Memoria delle istruzioni, PC, adder	23
4.2 Registri e ALU	23
4.3 Memoria dati ed unità di estensione del segno	24
4.4 Fetch della istruzione/aggiornamento del PC	24
4.5 Aggiungere i Jump	26
4.6 Segnali di controllo	27
5 Pipeline	29
5.1 Criticità (Hazard)	30
5.2 Propagazione (Forwarding)	30
5.3 Bolla (bubble)	31
5.4 Hazard sul controllo	31
5.5 Registri di pipeline	32
5.6 Realizzare il forwarding	34
5.7 Realizzare lo stallo	35
5.8 Anticipare i salti	36
5.9 Predire i salti	36
6 Cache	37
6.1 Direct mapping	37
6.2 Determinare HIT/MISS	38
6.3 Altri tipi di mapping	39
6.4 Cache multi-livello	40
7 Memoria virtuale	43
7.1 Mapping degli indirizzi virtuali	43
7.2 Supporto del Sistema Operativo	45

Introduzione al corso

L'obiettivo del corso "Architettura degli elaboratori" è quello di capire da cosa è formato un calcolatore ed il suo funzionamento in relazione alla comunicazione con il codice. Durante il corso, inoltre, conosceremo le strutture delle principali CPU moderne.

In particolar modo gli argomenti del corso saranno:

- Introduzione storica e struttura di una CPU;
- L'assembler MIPS;
- Progetto della CPU MIPS ad un colpo di clock;
- Introduzione alla pipeline;
- Progetto della CPU MIPS con pipeline;
- Gestione degli hazard ed eccezioni;
- Gerarchia di memoria e cache;
- Parallelismo con cache;
- Memoria virtuale e protezione.

L'esame sarà diviso in due parti: una prova pratica che riguarda la programmazione assembly e una prova scritta.

Gerarchia hardware-software

Una delle prime domande che ci poniamo è come è possibile tradurre da linguaggio di alto livello a linguaggio comprensibile dal calcolatore?

Ebbene come vedremo più avanti, il calcolatore può solo eseguire istruzioni di basso livello estremamente semplici. Per passare da un'applicazione complessa (scritta in un linguaggio vicino a quello dell'essere umano) alle semplici istruzioni che possono essere comprese dal calcolatore si utilizza un processo che coinvolge diversi strati di software.

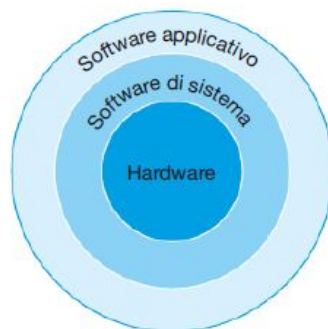


Figura 1: Gerarchia hardware-software

Questi strati sono organizzati principalmente in maniera gerarchica come mostrato in Figura 1. Nel cerchio più esterno compaiono le applicazioni, mentre i diversi componenti del software di sistema sono posizionati nel cerchio intermedio, a cui fanno parte il sistema operativo che permette di interfacciare i programmi utente con l'hardware del calcolatore e i compilatori che permettono la traduzione di un programma scritto in un linguaggio ad alto livello, in istruzioni eseguibili dall'hardware.

Architettura del calcolatore

Architettura dell'insieme di istruzioni (ISA): detta anche semplicemente architettura è un'interfaccia astratta tra l'hardware e il livello più basso del software di un calcolatore. Comprende tutte le informazioni necessarie per scrivere un programma in linguaggio macchina funzionante in modo corretto, comprese le istruzioni, i registri, gli accessi a memoria, l'I/O, l'unità di controllo (indica cosa fare in base alle istruzioni del programma), l'unità di elaborazione dei dati (effettua operazioni aritmetico-logiche) ecc.

Unità di misura e prestazioni della CPU

Decimale	Abb	Val	Binario	Abb	Val	%
Kylobyte	KB	10^3	Kibibyte	KiB	2^{10}	2%
Megabyte	MB	10^6	Mebibyte	MiB	2^{20}	5%
Gigabyte	GB	10^9	Gibibyte	GiB	2^{30}	7%
Terabyte	TB	10^{12}	Tebibyte	TiB	2^{40}	10%
Petabyte	PB	10^{15}	Pebibyte	PiB	2^{50}	13%
Exabyte	EB	10^{18}	Exbibyte	EiB	2^{60}	15%
Zettabyte	ZB	10^{21}	Zebibyte	ZiB	2^{70}	18%
Yottabyte	YB	10^{24}	Yobibyte	YiB	2^{80}	21%
Ronnabyte	RB	1000^9	Robibyte	RiB	2^{90}	24%
Queccabyte	QB	10^{10}	Quebibyte	QiB	2^{100}	27%

Tabella 1: Tabella delle unità di misura

Nella Tabella 1 sono riportate in forma decimale e binaria le unità di misura riferite ai calcolatori. Nell'ultima colonna, in percentuale, viene indicato quanto l'unità di misura binaria è più grande rispetto alla sua corrispondente decimale.

Grazie a queste unità di misura è possibile fare delle stime sul tempo di esecuzione della CPU.

$$Prestazioni = \frac{1}{\text{Tempo di esecuzione}}$$

Le metriche più elementari (numero di cicli di clock e periodo del clock) sono legate al tempo di CPU tramite una semplice equazione:

$$\text{Tempo di CPU relativo a un programma} = \frac{\text{Cicli di clock della CPU relativi al programma}}{\text{Periodo del clock}}$$

$$\text{Tempo di CPU relativo a un programma} = \frac{\text{Cicli di clock della CPU relativi al programma}}{\text{Frequenza di clock}}$$

Un ciclo di clock corrisponde ad un ciclo che partendo dall'elemento di stato, fa se sue operazioni e torna all'inizio. La rappresentazione numerica è basata sui valori di tensione del clock (alto o basso).

Quanto vale un ciclo di clock? Come abbiamo visto sopra, dipende dalla frequenza.

Il tempo di esecuzione di un programma non dipende quindi solo dalla frequenza del clock ma anche da:

$$\text{Cicli di clock della CPU} = \frac{\text{Numero di istruzioni del programma}}{\text{Numero medio di cicli clock per istruzione}}$$

1. Numero di istruzioni di un programma;
2. CPI (media di clock per istruzione), in quanto ogni istruzione ha un proprio peso;
3. Frequenza del clock.

Principi di progettazione

Con principi di progettazione dell'hardware intendiamo tutte quelle "best practice" che utilizziamo per una migliore resa del prodotto finale:

1. La semplicità favorisce la regolarità;
2. Minori sono le dimensioni, maggiore è la velocità;
3. Un buon progetto richiede buoni compromessi.

Parte I

Introduzione alla CPU

Capitolo 1

Architettura di Von Neumann

Il modello di von Neumann rappresenta in modo molto schematico le principali componenti hardware di un computer e le loro interconnessioni. Questa architettura è divisa in:

- **CPU** (Central Processing Unit o processore). E' formata da ALU (Unità Aritmetica Logica) che contiene tutti i circuiti per eseguire le operazioni aritmetico-logiche e la CU (o unità di controllo) per l'esecuzione dei programmi;
- **Memoria**. Nell'architettura dei calcolatori, si distinguono due tipi di memoria: la memoria centrale primaria, che lavora a diretto contatto con il processore e la memoria secondaria o di massa;
- **Periferiche di Input/Output**. Sono tutti quei dispositivi che non appartengono alla CPU o alla memoria (tastiera, schermo, stampante, scheda di rete ecc.);
- **Bus**. La CPU, la memoria e i dispositivi di Input/Output si scambiano i dati attraverso il Bus. Il Bus è l'insieme di linee su cui viaggiano i segnali elettrici formati da 0 e 1.

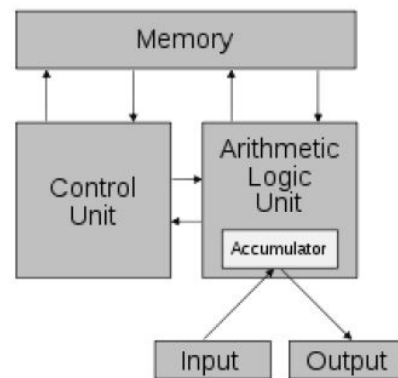


Figura 1.1: Architettura di Von Neumann

Capitolo 2

Assembler MIPS

2.1 Architetture CISC e RISC

Possiamo distinguere due macro categorie di architetture, che si basano su due filosofie diverse:

- **CISC** (Complex Instruction Set Computer), sono le prime a nascere ma sono allo stesso tempo le più complesse (contengono più microistruzioni) ed inoltre si basano sull'accesso in memoria (quindi anche gli indirizzamenti saranno adattati a questa filosofia).
- **RISC** (Reduced Instruction Set Computer), nascono successivamente. Le istruzioni sono poco complesse ed inoltre si basano sui registri. La filosofia è quella di far parlare più istruzioni in meno tempo possibile. Non è realmente importante la velocità di esecuzione delle istruzioni stesse.

2.2 MIPS

Il **MIPS** (Microprocessor without Interlocked Pipelined Stages) è di tipo RISC. Nel MIPS gli operandi devono essere contenuti nei registri per potere eseguire delle operazioni.

Alla memoria si accede solamente attraverso le istruzioni di trasferimento dati. Il MIPS utilizza l'indirizzamento al byte, perciò due parole consecutive hanno indirizzi in memoria a una distanza di 4. La memoria consente di memorizzare strutture dati, vettori, o il contenuto dei registri.

Tramite il linguaggio assembler MIPS possiamo svolgere diverse operazioni:

2.3 Rappresentazione dell'istruzione

Le istruzioni della CPU sono formate tutte da 32 bit con formato molto simile. Distinguiamo i tipi di rappresentazione in tre formati in base al numero di registri utilizzati e alle operazioni svolte:

- **R-type** istruzioni senza accesso alla memoria e con istruzioni aritmetico\logiche;
- **I-type** istruzioni di trasferimento immediato;
- **J-type** istruzioni di salto.

R-type

Con la rappresentazione R-type occupiamo 6 bit per l'opcode, che rappresenta la classe di operazioni. 5 bit per l'rs, ovvero il primo registro dell'operazione e la stessa cosa avviene con rt, il secondo registro dell'operazione e rd il registro destinazione. 5 bit anche per lo shamt o shift amount, indice il valore dello shift. Infine 6 bit per il funct o function code, rappresenta il codice dell'operazione da svolgere.

op	rs	rt	rd	shamt	funct
----	----	----	----	-------	-------

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Operandi in tre registri
	Sottrazione	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Operandi in tre registri
	Somma immediata	addi \$s1,\$s2,20	$\$s1 = \$s2 + 20$	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola	lw \$s1,20(\$s2)	$\$s1 = \text{Memoria}[\$s2+20]$	Trasferimento di una parola da memoria a registro
	Memorizzazione parola	sw \$s1,20(\$s2)	$\text{Memoria}[\$s2+20] = \$s1$	Trasferimento di una parola da registro a memoria
	Lettura mezza parola	lh \$s1,20(\$s2)	$\$s1 = \text{Memoria}[\$s2+20]$	Trasferimento di una mezza parola da memoria a registro
	Lettura mezza parola, senza segno	lhu \$s1,20(\$s2)	$\$s1 = \text{Memoria}[\$s2+20]$	Trasferimento di una mezza parola da memoria a registro
	Memorizzazione mezza parola	sh \$s1,20(\$s2)	$\text{Memoria}[\$s2+20] = \$s1$	Trasferimento di una mezza parola da registro a memoria
	Lettura byte	lb \$s1,20(\$s2)	$\$s1 = \text{Memoria}[\$s2+20]$	Trasferimento di un byte da memoria a registro
	Lettura byte senza segno	lbu \$s1,20(\$s2)	$\$s1 = \text{Memoria}[\$s2+20]$	Trasferimento di un byte da memoria a registro
	Memorizzazione byte	sb \$s1,20(\$s2)	$\text{Memoria}[\$s2+20] = \$s1$	Trasferimento di un byte da registro a memoria
	Lettura di una parola e blocco	ll \$s1,20(\$s2)	$\$s1 = \text{Memoria}[\$s2+20]$	Caricamento di una parola come prima fase di un'operazione atomica
	Memorizzazione condizionata di una parola	sc \$s1,20(\$s2)	$\text{Memoria}[\$s2+20] = \$s1$; $\$s1 = 0$ oppure 1	Memorizzazione di una parola come seconda fase di un'operazione atomica
	Caricamento costante nella mezza parola superiore	lui \$s1,20	$\$s1 = 20 * 2^{16}$	Caricamento di una costante nei 16 bit più significativi
Logiche	And	and \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$	Operandi in tre registri; AND bit a bit
	Or	or \$s1,\$s2,\$s3	$\$s1 = \$s2 \mid \$s3$	Operandi in tre registri; OR bit a bit
	Nor	nor \$s1,\$s2,\$s3	$\$s1 = \sim(\$s2 \mid \$s3)$	Operandi in tre registri; NOR bit a bit

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Logiche (segue)	And immediato	andi \$s1,\$s2,20	$\$s1 = \$s2 \& 20$	And bit a bit tra un operando in registro e una costante
	Or immediato	ori \$s1,\$s2,20	$\$s1 = \$s2 \mid 20$	OR bit a bit tra un operando in registro e una costante
	Scorrimento logico a sinistra	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	Spostamento a sinistra del numero di bit specificato dalla costante
	Scorrimento logico a destra	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	Spostamento a destra del numero di bit specificato dalla costante
Salti condizionati	Salta se uguale	beq \$s1,\$s2,25	Se $(\$s1 == \$s2)$ vai a PC+4+100	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	bne \$s1,\$s2,25	Se $(\$s1 \neq \$s2)$ vai a PC+4+100	Test di disuguaglianza; salto relativo al PC
	Poni uguale a 1 se minore	slt \$s1,\$s2,\$s3	Se $(\$s2 < \$s3)$ $\$s1 = 1$; altrimenti $\$s1 = 0$	Comparazione di minoranza; utilizzata con bne e beq
	Poni uguale a uno se minore, numeri senza segno	sltu \$s1,\$s2,\$s3	Se $(\$s2 < \$s3)$ $\$s1 = 1$; altrimenti $\$s1 = 0$	Comparazione di minoranza su numeri senza segno
	Poni uguale a uno se minore, immediato	slti \$s1,\$s2,20	Se $(\$s2 < 20)$ $\$s1 = 1$; altrimenti $\$s1 = 0$	Comparazione di minoranza con una costante
	Poni uguale a uno se minore, immediato e senza segno	sltiu \$s1,\$s2,20	Se $(\$s2 < 20)$ $\$s1 = 1$; altrimenti $\$s1 = 0$	Comparazione di minoranza con una costante, con numeri senza segno
Salti incondizionati	Salto incondizionato	j 2500	Vai a 10000	Salto all'indirizzo della costante
	Salto indiretto	jr \$ra	Vai all'indirizzo contenuto in \$ra	Salto all'indirizzo contenuto nel registro, utilizzato per il ritorno da procedura e per i costrutti switch
	Salta e collega	jal 2500	$\$ra = \text{PC}+4$; vai a 10000	Chiamata a procedura

Ad esempio effettuando *add \$t0, \$s1, \$s2* otteniamo:

000000	10001	10010	01000	00000	100000
--------	-------	-------	-------	-------	--------

Notiamo che i registri da \$s0 a \$s7 corrispondono agli indirizzi 16-23 mentre i registri da \$t0 a \$t7 corrispondono agli indirizzi 8-15.

I-type

Con la rappresentazione I-type vengono rimpiazzati i 16 bit di rd, shamt e funct con constant or address che conterrà un totale di 16 bit:

op	rs	rt	constant or address
----	----	----	---------------------

J-type

Le uniche istruzioni di salto sono le *j* e *jal*. Con la rappresentazione J-type utilizziamo solamente l'op e l'indirizzo di destinazione:

op	address
----	---------

2.4 Tipi di indirizzamento alla memoria

Organizzazione della memoria

Dobbiamo immaginare la memoria come una pila di elementi indirizzati. Gli indirizzi di memoria vengono contati a multipli di quattro. Partendo da queste precisazioni seguiamo lo schema proposto in Figura 2.1, dove il PC (Program Counter) è un registro contatore la cui funzione è quella di conservare l'indirizzo di memoria della prossima istruzione da eseguire. Il GP (Global Pointer) è un registro usato per accedere tramite un offset alla sezione data. Lo SP (Stack Pointer) è un registro che contiene l'indirizzo di inizio dello stack.

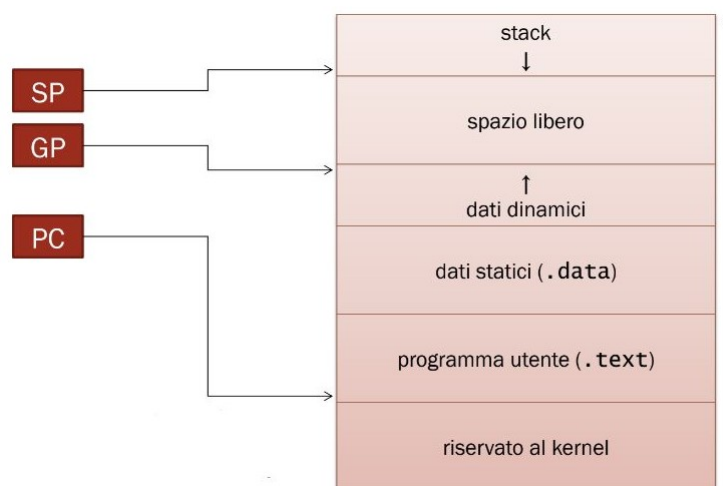


Figura 2.1: Organizzazione della memoria

Cosa sa fare la CPU?

Con l'IAS Machine abbiamo visto tutte le fasi di esecuzione di un'istruzione. Le tipologie di istruzioni che la CPU può svolgere sono:

- **LOAD\ STORE.** Trasferiscono dati da/verso la memoria. Es: *lw, lh, lb, sw, sh, sb, lwc1, swc1*.

- **Logico\ Aritmetiche.** Svolgono i calcoli aritmetici e logici Es: add, sub, mul, div, sll, srl, sra.
- **Salti condizionati e incondizionati.** Controllano il flusso logico di esecuzione. Es: j, jal, brqz, blt, ble.
- **Gestione delle eccezioni/interrupt.** Salvataggio dello stato e suo ripristino. Es: eret.
- **Istruzioni di trasferimento dati.**

Codifica delle istruzioni

La codifica della istruzione indica quale operazione deve essere eseguita, tramite l'opcode, e successivamente inserire il risultato di tale operazione tramite l'address o altri registri.

Esistono principalmente due tipi di indirizzamento. Il modo diretto guida direttamente alla cella di memoria. Il modo indiretto, invece, fa uso di un altro indirizzo che contiene un puntatore alla zona di memoria dove verrà inserito il risultato. Esistono anche altre due varianti di questi processi con i registri, dove nel primo caso il risultato viene direttamente inserito nel registro, mentre nel secondo il registro contiene l'indirizzo di memoria dell'address.

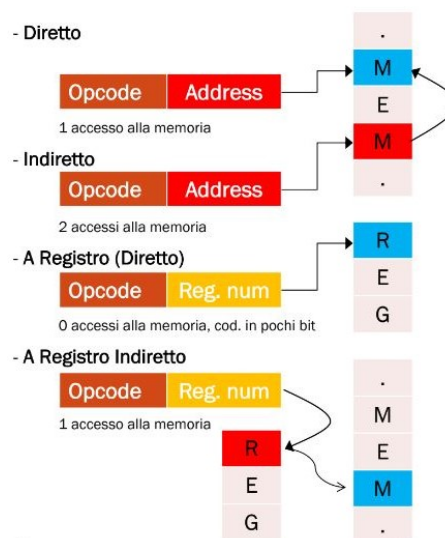


Figura 2.2: Indirizzamenti diretti e indiretti

Esiste anche un modo più intelligente utilizzando lo spiazzamento.

Questo metodo è più efficiente in quanto con quelli precedenti siamo obbligati a sommare e sottrarre indirizzi di memoria. Con questo tipo di indirizzamento tutto ciò non avviene perché utilizziamo un registro dove viene inserito il valore interessato, che verrà calcolato successivamente. R rappresenta infatti il registro dove si trova l'accesso alla memoria mentre l'offset ci indica di quanto dobbiamo spostarci. Con un esempio sarà tutto più chiaro.

Esempio. Mi trovo ad indirizzo 0, 0xAAAB1234, e voglio arrivare alla terza riga della pila, ovvero 8 (dato che gli indirizzi vengono contati di 4 in 4).

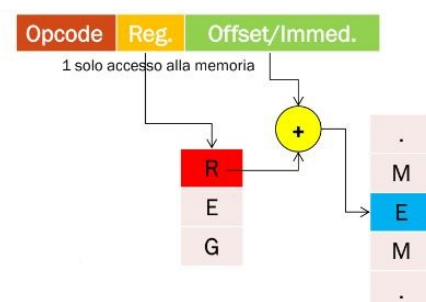
Con i metodi visitati precedentemente dovrei fare:

$$0xAAAB1234 + 8 = 0xAAAB123C$$

e per tornare all'inizio dovrei sottrarre di nuovo 8. Mentre con lo spiazzamento effettuo:

$$0xAAAB1234 + \$s0 \times 4$$

e in \$s0 inserisco il valore 2. Così ottengo $2 \times 4 = 8$.



MIPS 2000

L'architettura MIPS 2000 è formata da 3 CPU:

- Una prima CPU formata da 32 registri da 32 bit. Viene utilizzata per la moltiplicazioni e divisioni.
- Il coprocessore 0 che si occupa della memorizzazione e della cattura delle eccezioni.
- Il coprocessore 1 che viene utilizzato per i calcoli in virgola mobile.

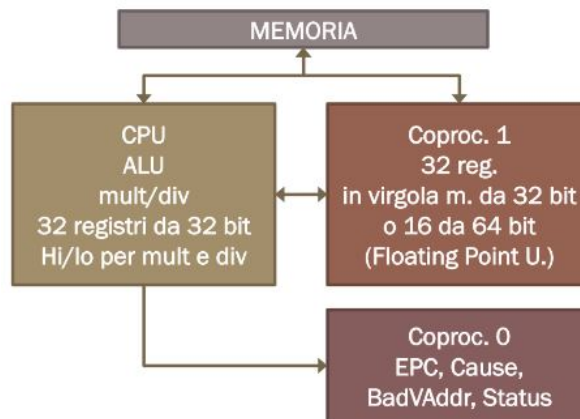


Figura 2.3: MIPS 2000

I 32 registri della CPU sono divisi come segue:

- \$zero è un registro immutabile che contiene la costante zero;
- \$at usato dalle pseudoistruzioni;
- \$v0, \$v1 dove v sta per value. Tenzionalmente vengono inseriti i risultati delle funzioni;
- \$a0 . . \$a3 vengono utilizzati per contenere gli argomenti delle funzioni;
- \$t0 . . \$t7 sono registri temporanei salvati dal chiamante;
- \$s0 . . \$s7 sono temporanei salvati dal chiamato;
- \$t8, \$t9 altri registri temporanei salvati dal chiamante;
- \$k0, \$k1 kernel per eccezioni;
- \$gp global pointer (memoria dinamica);
- \$sp stack pointer (stack delle funzioni);
- \$fp frame pointer (funzioni);
- \$ra return address.

Parte II

Progettare la CPU

Capitolo 3

Elaborare un'istruzione

Le fasi di esecuzione di un'istruzione sono:

- **caricamento** di una istruzione dalla memoria alla CU;
- **decodifica** della istruzione e lettura argomenti dai registri;
- **esecuzione** (attivazione delle unità funzionali necessarie);
- **accesso** alla memoria;
- **scrittura** dei risultati nei registri;
- **aggiornamento** del PC (normale/salti condizionati/salti non condizionati).

Le istruzioni che la CPU può eseguire sono: accesso alla memoria: `lw`, `sw` (di tipo I); salti condizionati: `beq` (di tipo I); operazioni aritmetico-logiche: `add`, `sub`, `sll`, `slt`, ... (di tipo R); salti non condizionati `j`, `jal` (di tipo J); operazioni con costanti `li`, `addi`, `subi`, ... (di tipo I).

Ricordiamo il formato delle tre istruzioni MIPS (r, i, j type) codificate nella memoria:

Nome	Campi						Commenti
Dimensione del campo	6 bit	5 bit	5 bit	5 bit	5 bit	6 bit	Tutte le istruzioni MIPS sono a 32 bit
Formato R	op	rs	rt	rd	shamt	funct	Formato delle istruzioni aritmetiche
Formato I	op	rs	rt	indirizzo / costante			Formato delle istruzioni di trasferimento dati di salto condizionato e immediate
Formato J	op	indirizzo di destinazione					Formato delle istruzioni di salto incondizionato

Capitolo 4

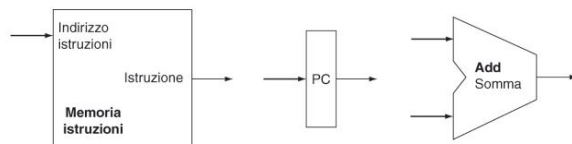
Unità della CPU

4.1 Memoria delle istruzioni, PC, adder

La *memoria istruzioni* riceve in input un indirizzo da 32 bit e restituisce in output l'istruzione (opcode) situata nell'indirizzo di input.

Il *program counter* o PC è un registro che contiene l'indirizzo dell'istruzione corrente.

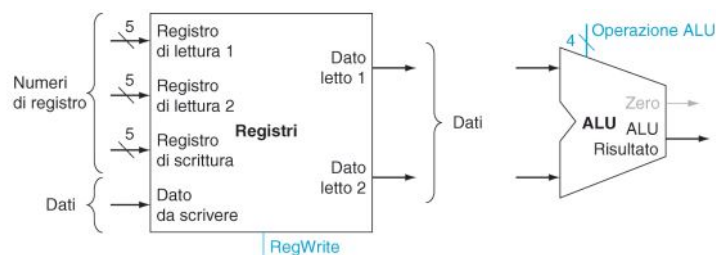
Tramite l'*adder*, se necessario, si calcolerà la nuova destinazione del program counter, per leggere una successiva istruzione o per un salto relativo.



4.2 Registri e ALU

Il *blocco dei registri* contiene 32 registri a 32 bit, indirizzabili tramite 5 bit ($2^5 = 32$). E' formato da 3 porte da 5 bit per indicare quali registri leggere (*rs*, *rt*) e in quale scrivere (*rd*), 3 porte dati (a 32 bit), di cui una in ingresso per il valore da memorizzare e 2 di uscita per i valori letti (capiremo dopo il perchè di questa doppia scelta) e il segnale *RegWrite* che abilita (se 1) la scrittura nel registro di scrittura.

L'*ALU*, invece, riceve due valori interi a 32 bit e svolge l'operazione indicata dal segnale *Op . ALU*. Oltre al risultato da 32 bit produce un segnale *Zero* riportato se il risultato è zero.



Notiamo che i bit dell'*Op . ALU*, dagli originali 6, si riducono a 4. Ciò avviene perchè è stato stabilito che bastano solamente 4 bit per rappresentare le operazioni che l'ALU può svolgere.

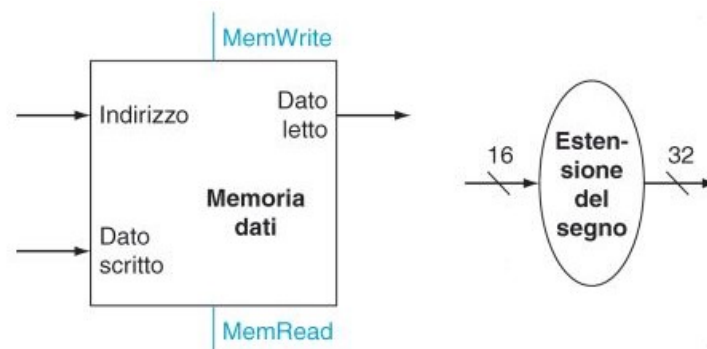
ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

Figura 4.1: Operazioni dell'ALU

4.3 Memoria dati ed unità di estensione del segno

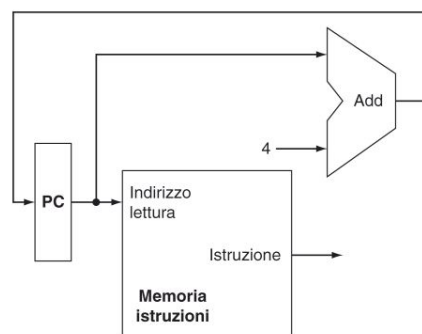
L'*unità di memoria* viene utilizzata per tutte quelle operazioni che richiedono accessi alla memoria come lw , sw . L'unità di memoria riceve un indirizzo (da 32 bit) che indica quale word della memoria va letta/scritta, riceve il segnale $MemRead$ che abilita la lettura dall'indirizzo (se lw), riceve un dato da 32 bit da scrivere in memoria a quell'indirizzo (se sw), riceve il segnale di controllo $MemWrite$ che abilita (1) la scrittura del dato all'indirizzo.

L'*unità di estensione del segno* è di grande aiuto per quelle istruzioni che utilizzano 16 bit ma per l'operazione se ne richiedono 32. Nello specifico serve a trasformare un intero relativo (in CA2) da 16 a 32 bit.



4.4 Fetch della istruzione/aggiornamento del PC

L'*aggiornamento del program counter* viene effettuato tramite un adder seguendo vari passaggi. Nel PC si trova l'indirizzo dell'istruzione, l'istruzione viene letta, mentre avviene la lettura il PC viene incrementato di 4 bit (1 word) e il valore viene inserito nuovamente nel PC.



Esempio. Proviamo a rappresentare il percorso delle seguenti istruzioni con i moduli visti fin ora:

`add $s0, $s1, $s2; lw $s0, 4($s1); sw $s0, 4($s1); beq $s0, $s1, 0x00000014.` Ricordiamo che per prendere delle scelte utilizziamo i multiplexer. In particolare a seconda del segnale di controllo

ALUSrc viene selezionata la porta corrispondente del MUX di sinistra. Per calcolare l'indirizzo di accesso alla memoria si usa la stessa ALU. Il risultato dell'ALU o della lw viene selezionato da MemtoReg che comanda il MUX a destra.

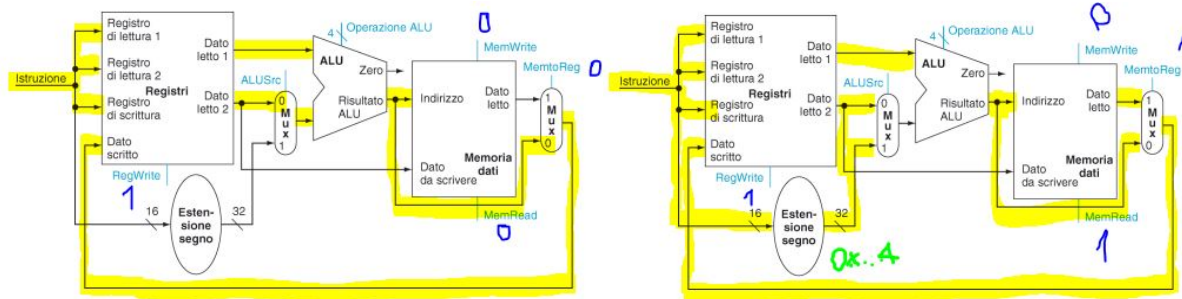


Figura 4.2: add \$s0, \$s1 \$s2 e lw \$s0, 4(\$s1)

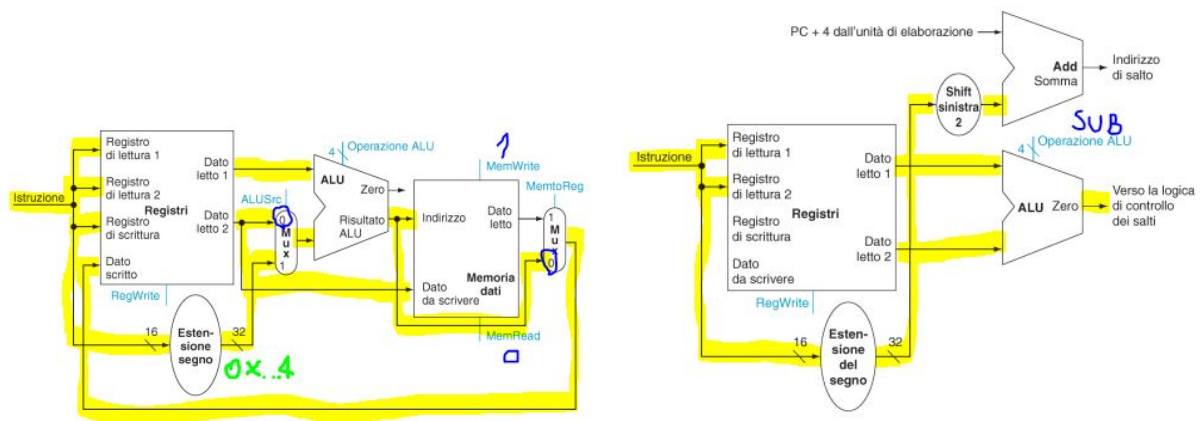


Figura 4.3: sw \$s0, 4(\$s1) e beq \$s0, \$s1, 0x00000014

CPU (quasi) completa

Con l'unione di tutti i moduli visti precedentemente, con l'aggiunta dei multiplexer come branch di scelte, Control Unit (il cervello della CPU che comanda le scelte da prendere) e la logica del salto (incrementa il program counter), otteniamo il seguente schema della CPU:

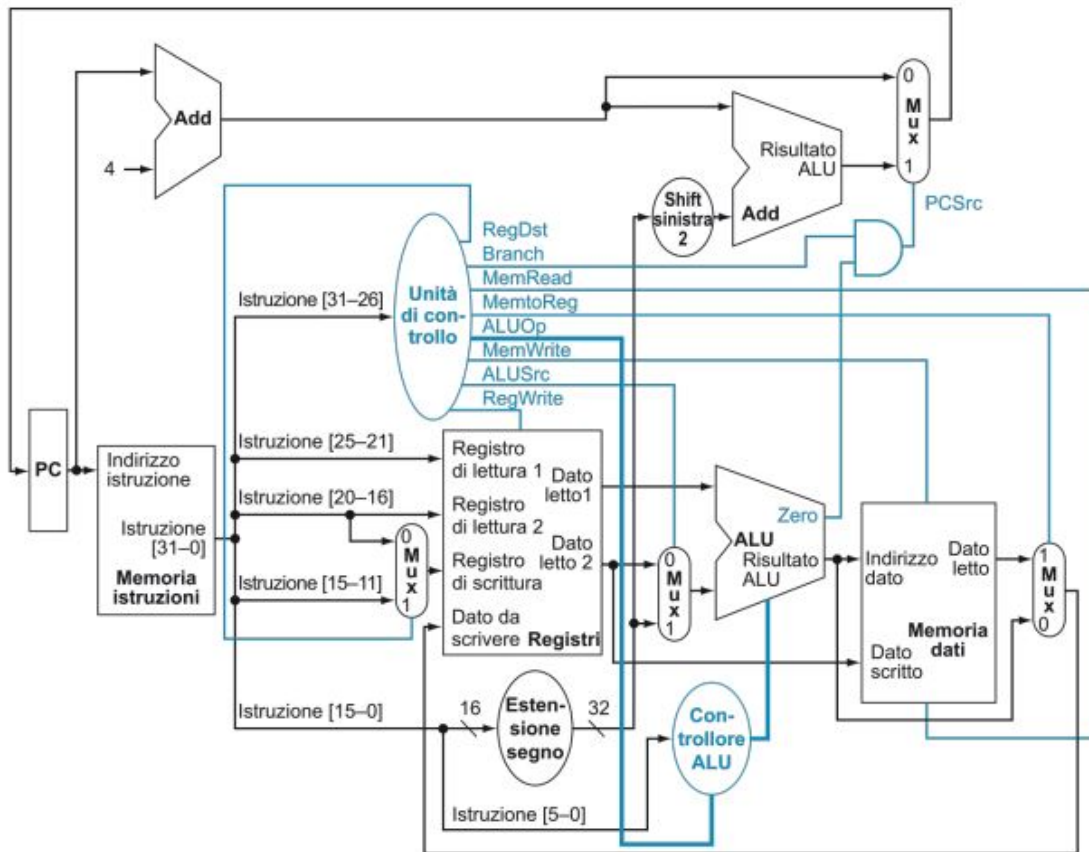


Figura 4.4: Schema della CPU

4.5 Aggiungere i Jump

Supponiamo di voler aggiungere la nuova istruzione, J (*Jump*). Ricordiamo la codifica dei salti (formato J):

000010	indirizzo
31-26	25:0

L'indirizzo di destinazione del salto è contenuto nei 26 bit (25:0), ed è un indirizzo assoluto, ovvero non relativo al PC come ad esempio per i branch. Per tanto questo indirizzo va moltiplicato per 4 perché le istruzioni sono allineate, così da ricavare un'istruzione di 28 bit. E i mancanti 4 bit (per ottenere un totale di 32 bit) verranno presi dal PC+4.

Nello specifico le operazioni da svolgere saranno le seguenti:

$$PC \leftarrow (\text{shift left di 2 bit di Istruzione}[25-0]) \text{ OR } (PC + 4)[31-28]$$

Allo schema della CPU non resta che aggiungere lo shift left di 2 bit con input a 26 bit e MUX per selezionare il nuovo PC (l'OR dei 28 bit ottenuti con i 4 del PC+4 si ottiene dalle connessioni). E' importante ricordarsi di settare i controlli RegWrite e MemWrite a 0 per evitare modifiche a registri e alla memoria. [Vedi Figura 4.5]

L'istruzione Jal (*Jump And Link*) è sempre codificata come tipo J, inoltre le operazioni preliminari sono le stesse del Jump con l'aggiunta del comando \$ra <- PC+4.

Le unità funzionali previste saranno le stesse di quelle utilizzate per il Jump, con l'aggiunta di un MUX per selezionare il valore di PC+4 come valore di destinazione e un MUX per selezionare il numero del registro \$ra come destinazione. Infatti il primo di questi MUX conterrà il PC+4, mentre il secondo il valore 31 che rappresenta il 31° registro del MIPS che è esattamente il registro \$ra. La

nuove combinazioni. Una volta individuate le nuove funzionalità (fallaci) delle istruzioni possiamo cercare di scrivere un breve programma che evidenzia se la CPU sia mal funzionante o meno.

Capitolo 5

Pipeline

Nel capitolo precedente abbiamo analizzato il processo di elaborazione di un'istruzione. Per semplicità possiamo riassumere le varie fasi dell'istruzione in:

- **Fetch:** carica l'istruzione dalla memoria;
- **Instruction Decode:** la CU decodifica l'istruzione e vengono letti gli argomenti dai registri;
- **Execution:** l'ALU fa il calcolo necessario (tipo R o accesso alla memoria o branch);
- **Memory access:** viene letta/scritta la memoria (lw e sw);
- **Write Back:** il risultato dell'ALU o quello letto dalla memoria viene messo nel registro dest.

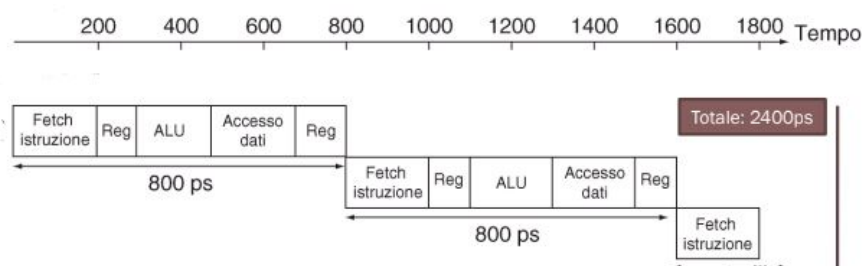
Notiamo come eseguendo un'istruzione per volta, la CPU è per l'80% inutilizzata. Da qui nasce l'idea di utilizzare le *pipeline* ovvero dei processi per incrementare il throughput, ovvero la quantità di istruzioni eseguite in una data quantità di tempo, parallelizzando i flussi di elaborazione di più istruzioni. In altre parole vogliamo ottimizzare i processi della CPU rendendo ogni unità funzionale autonoma di passare all'istruzione successiva. In questo modo possono essere eseguite fino a cinque istruzioni contemporaneamente nel caso migliore (anche se noteremo che non sempre sarà possibile).

Istruzione 1	IF	ID	EXE	MEM	WB				
Istruzione 2		IF	ID	EXE	MEM	WB			
Istruzione 3			IF	ID	EXE	MEM	WB		
Istruzione 4				IF	ID	EXE	MEM	WB	
Istruzione 5					IF	ID	EXE	MEM	WB

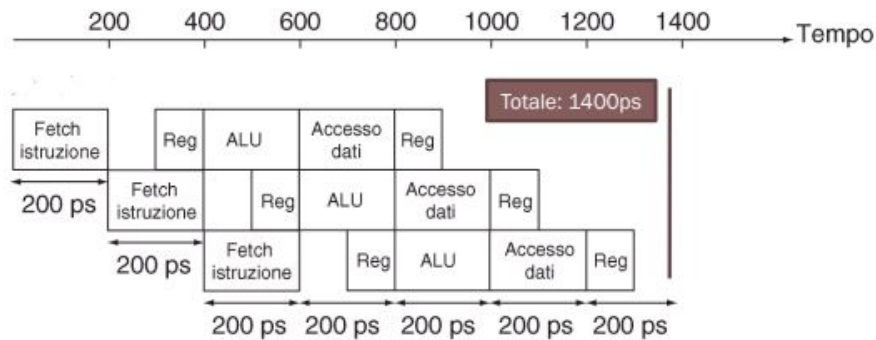
Figura 5.1: Cinque istruzioni ottimizzate con le pipeline

Ad ogni colpo di clock una istruzione viene completata dalla pipeline. Possiamo arrivare a quintuplicare (idealmente) il throughput.

Esempio. Analizziamo i risultati delle seguenti istruzioni lw \$1,100(\$0) lw \$2,200(\$0) lw \$3,300(\$0).
Con un approccio sequenziale delle istruzioni otterremmo il seguente risultato:



Mentre con le pipeline:



Ogni componente lavora in maniera autonoma, senza aspettare il risultato dell'istruzione precedente (non sempre è possibile), in modo da passare all'istruzione successiva e migliorare i tempi di elaborazione.

E' possibile utilizzare in maniera efficiente le pipeline nell'architettura MIPS perchè di tipo RISC (vedi 2.1) infatti, con questo tipo di struttura, si presentano numerosi vantaggi come: i registri sorgenti sono nella stessa posizione, la lettura di *rs* viene fatta durante la fase ID, gli operandi in memoria solo utilizzati solo per *lw* e *sw*, il trasferimento del risultato avviene sempre in coda, è necessario un singolo accesso in memoria per dato.

5.1 Criticità (Hazard)

Ovviamente non è sempre possibile sfruttare questi vantaggi in maniera efficiente. Si possono presentare dei casi in cui:

- le risorse hardware non sono sufficienti (*structural hazard*);
- il dato necessario non è ancora pronto (*data hazard*);
- la presenza di un salto cambia il flusso di esecuzione delle istruzioni (*control hazard*).

Esempio. Le criticità più frequenti sono quelle legate ai dati. Ad esempio `add $s0,$t0,$t1 sub $t2,$s0,$t3`.



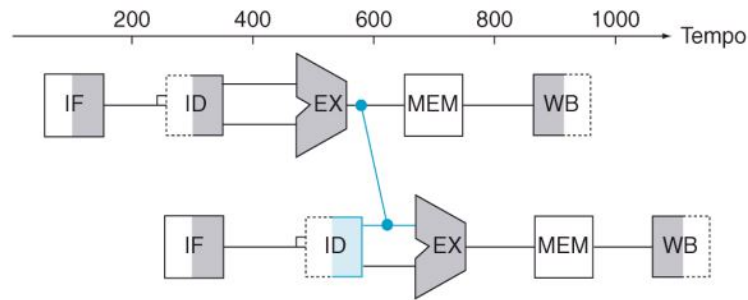
Non può essere eseguita la lettura degli elementi della seconda istruzione finchè non si esegue il write back. Inoltre si possono inserire in parallelo il WB e l'ID perchè a livello di architettura viene eseguito prima il write e dopo il read (proprio per risparmiare tempo in questo tipo di casi).

5.2 Propagazione (Forwarding)

In alcuni casi l'informazione necessaria è già presente nella pipeline, prima del WB. Possiamo, in questi casi inserire nel datapath delle "scorciatoie" che recapitano il dato all'unità funzionale che ne ha bisogno senza aspettare la fase di WB.

E' possibile utilizzare la propagazione solo quando lo stadio che deve ricevere il dato è successivo a quello che lo produce nel diagramma temporale della pipeline.

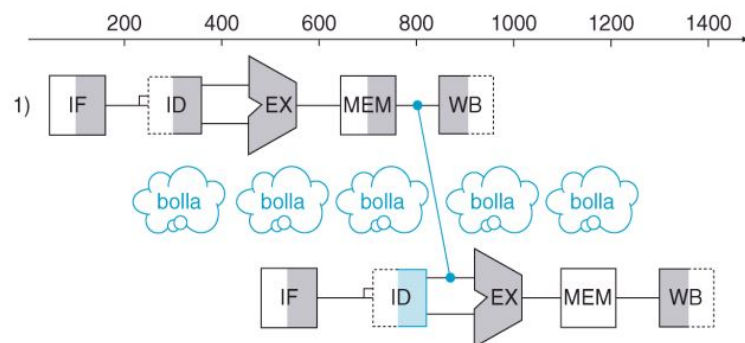
Riprendendo l'esercizio precedente possiamo creare un collegamento tra EX e ID per lavorare direttamente sul dato interessato:



5.3 Bolla (bubble)

Se la fase che ha bisogno del dato si trova prima (nel tempo) di quella che lo produce sarà necessario inserire una attesa (stallo o bolla). Una bolla è un'istruzione che non fa niente e deve essere inserita perchè la CPU non può produrla o aspettare lo stallò. Anche se è possibile utilizzare una bolla è sempre meglio inserire altre istruzioni successive per guadagnare in termini di efficienza.

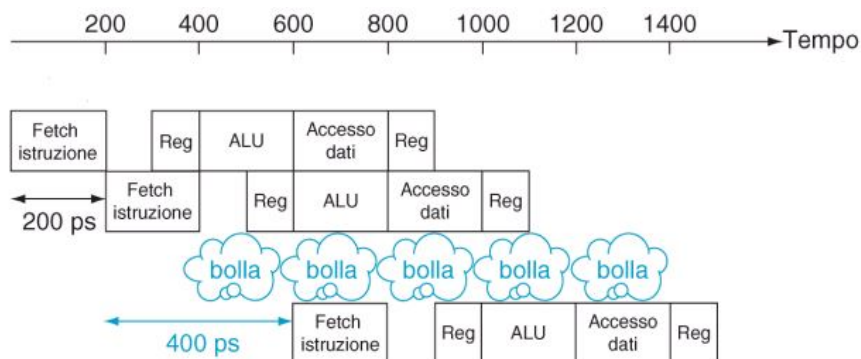
Esempio. Con le istruzioni `lw $s0, 20($t1)` `sub $t2, $s0, $t3` siamo obbligati ad utilizzare una bolla perchè per la seconda istruzione ci serve il valore memorizzato nella prima:



5.4 Hazard sul controllo

Quando abbiamo a che fare con istruzioni di controllo, l'istruzione successiva può essere caricata solo dopo che il salto è stato deciso. Sono inoltre necessari 2 stalli se la beq viene decisa in EXE o 1 se viene decisa in ID.

Esempio. Consideriamo queste tre istruzioni `add $4, $5, $6` `beq $1, $240 or $7, $8, $9`:



Per diminuire i control hazard possiamo:

- *Anticipare la decisione*: se la beq viene calcolata nell'ALU, nella fase EXE ci saranno 2 istruzioni da "buttare", se invece la beq viene decisa nella fase ID è sufficiente "buttare" 1 istruzione;
- *Ritardare il salto*: se l'istruzione che segue la beq viene sempre eseguita anche se viene effettuato il salto, si elimina lo stallo;
- *Branch prediction*: la CPU osserva i salti eseguiti e cerca di pre-caricare l'istruzione (seguito o di destinazione) eseguita più spesso.

5.5 Registri di pipeline

Riprendiamo ciò che abbiamo visto precedentemente con questa semplice immagine:

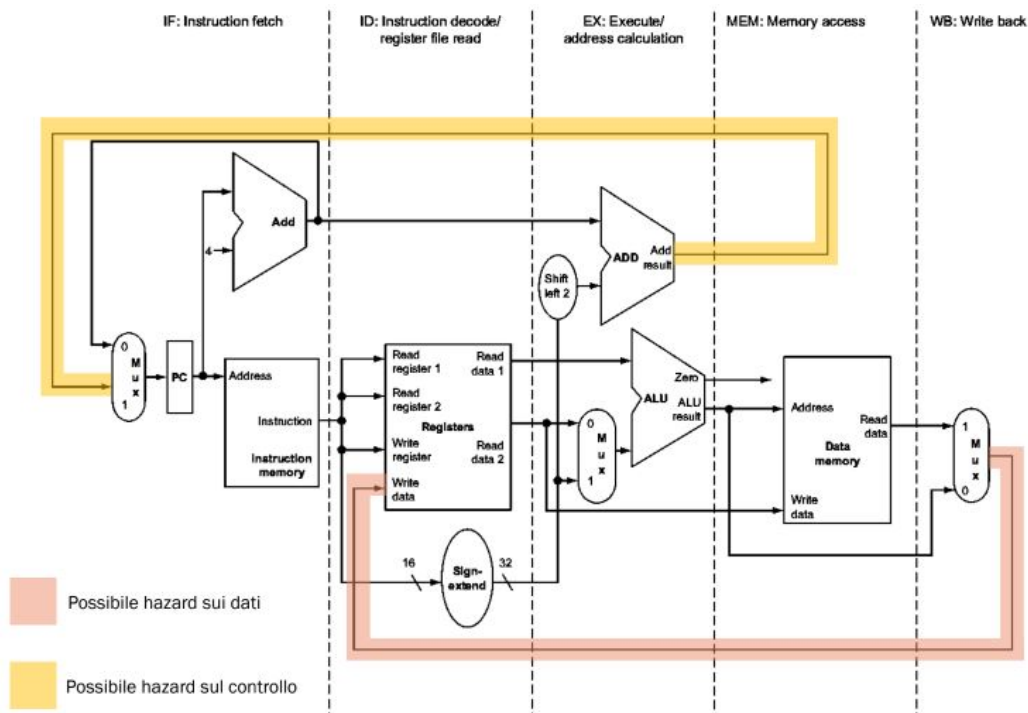


Figura 5.2: Unità funzionali nelle cinque fasi

In molti casi si è presentata la necessità di utilizzare i valori contenuti nelle fasi precedenti per svolgere correttamente più istruzioni alla volta. In tal senso vengono in nostro aiuto i *registri di pipeline* che ci permettono di salvare, in registri di appoggio, il contenuto dopo ogni fine fase.

I registri di pipeline sono in totale quattro, perchè sull'ultima unità funzionale (Write Back) non servono valori da memorizzare in quanto questi verranno direttamente scritti nel registro destinazione. Inoltre questi registri hanno dimensioni diverse tra di loro in base alla quantità di informazioni che devono memorizzare all'uscita dell'unità funzionale.

Per evitare di sovrascrivere i registri delle unità funzionali durante l'esecuzione in pipeline di altre istruzioni, dobbiamo salvare il valore del registro di destinazione e propagare i segnali di controllo. Possiamo dividere i segnali di controllo delle fasi esecutive in tre gruppi: fase EXE, fase MEM, fase WB:

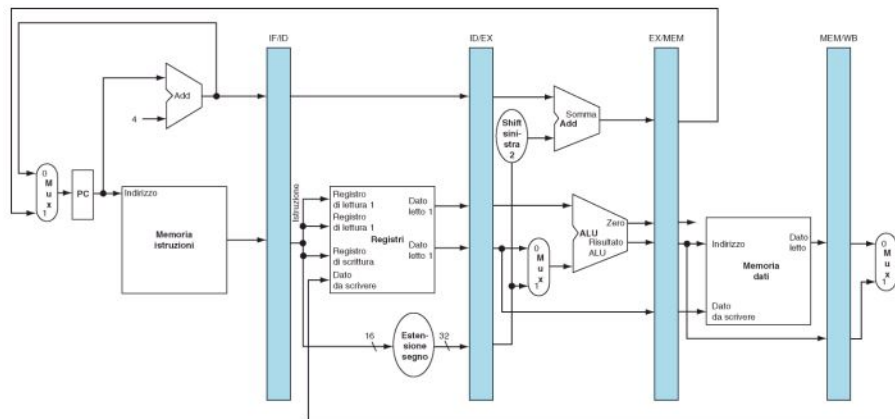


Figura 5.3: Registri di pipeline

Istruzione	Segnali di controllo dello stadio di esecuzione/calcolo dell'indirizzo				Segnali di controllo dello stadio di accesso alla memoria dati			Segnali di controllo dello stadio di Write-back	
	RegDst	ALUOp1	ALUOp0	ALUSrc	Branch Mem	Read Mem	Mem-Write	RegWrite	Memto-Reg
Formato-R	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

Figura 5.4: Segnali di controllo delle pipeline

Questi segnali devono essere passati di fase in fase nei registri della pipeline. Cosicché ad ogni fase salviamo i dati (letti dai registri o dalla parte immediata) ed i segnali di controllo della stessa istruzione (generati dalla CU):

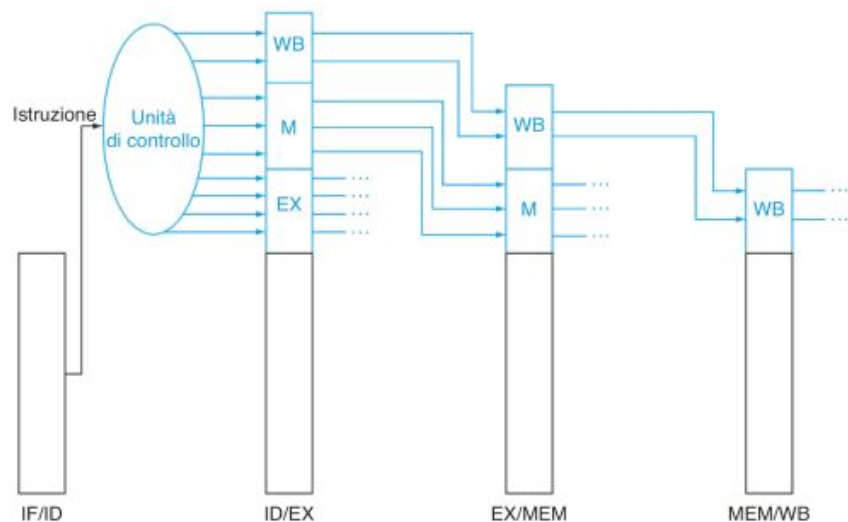


Figura 5.5: Propagazione dei segnali di controllo in pipeline

Ora possiamo finalmente dare uno sguardo allo schema della CPU con l'aggiunta delle pipeline, anche se nei prossimi paragrafi verrà leggermente aggiornato:

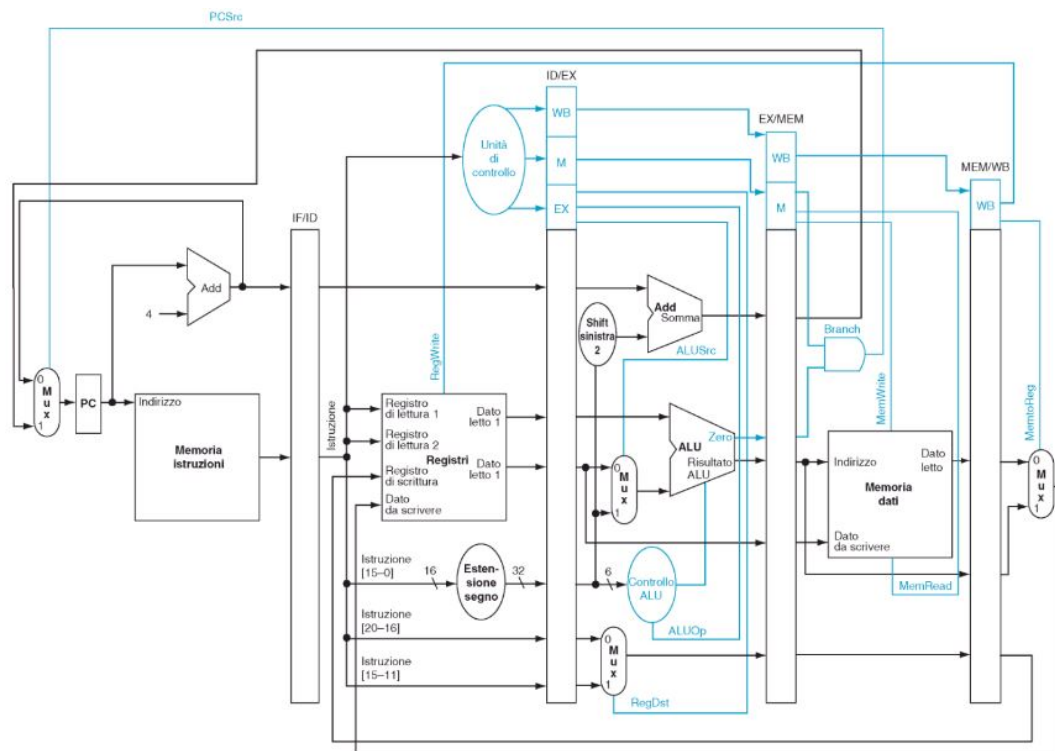


Figura 5.6: CPU con pipeline

5.6 Realizzare il forwarding

Torniamo a parlare di forwarding, e in particolare modo di quello in EXE, ovvero quel processo che ci permette di sostituire il valore letto dal blocco registri con quello prodotto dall'istruzione precedente.

Con la struttura della CPU vista finora, per realizzare il forwarding, vogliamo apportare delle modifiche al datapath, ovvero inserire un MUX prima dell'ALU per distinguere tre casi:

1. *Non c'è forwarding:* il valore per la ALU viene dal registro ID/EX della pipeline;
2. *Forwarding dall'istruzione precedente:* il valore per la ALU viene dal registro EX/MEM della fase precedente;
3. *Forwarding dalla seconda istruzione precedente:* il valore per la ALU viene dal registro MEM/WB della fase precedente.

Quindi per realizzare il forwarding dobbiamo inserire due MUX prima dell'ALU (uno per il registro rs e uno per rt) e creare dei segnali per il forwarding:

Controllo multiplexer	Sorgente	Spiegazione
PropagaA = 00	ID/EX	Il primo operando della ALU proviene dal register file.
PropagaA = 10	EX/MEM	Il primo operando della ALU viene propagato dal risultato della ALU nel ciclo di clock precedente.
PropagaA = 01	MEM/WB	Il primo operando della ALU viene propagato dalla memoria dati o da un precedente risultato della ALU.
PropagaB = 00	ID/EX	Il secondo operando della ALU proviene dal register file.
PropagaB = 10	EX/MEM	Il secondo operando della ALU viene propagato dal risultato della ALU nel ciclo di clock precedente.
PropagaB = 01	MEM/WB	Il secondo operando della ALU è propagato dalla memoria dati o da un precedente risultato della ALU.

Figura 5.7: Segnali per il forwarding

Nota. E' possibile realizzare il forwarding anche nella fase ID (necessario SOLO se la beq viene anticipata in ID) e nella fase MEM (necessario SOLO se lw \$rt, ... è subito seguita da sw \$rt, ...).

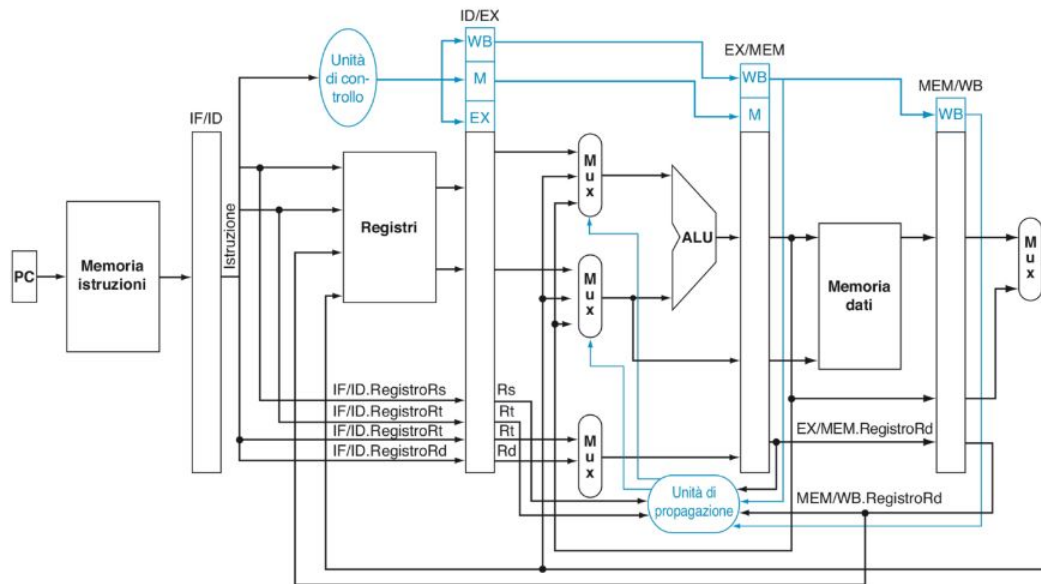


Figura 5.8: CPU con forwarding

5.7 Realizzare lo stallo

Come abbiamo visto in precedenza talvolta l'istruzione deve attendere che sia pronto il dato perché possa avvenire il forwarding.

Per fermare l'istruzione con uno stallo, dobbiamo (nella fase ID) annullare l'istruzione che deve attendere (bolla), azzerando i segnali di controllo MemWrite e RegWrite nonché IF/ID. Istruzione e rileggere la stessa istruzione affinché possa essere rieseguita un ciclo di clock dopo impedendo quindi che il PC si aggiorni.

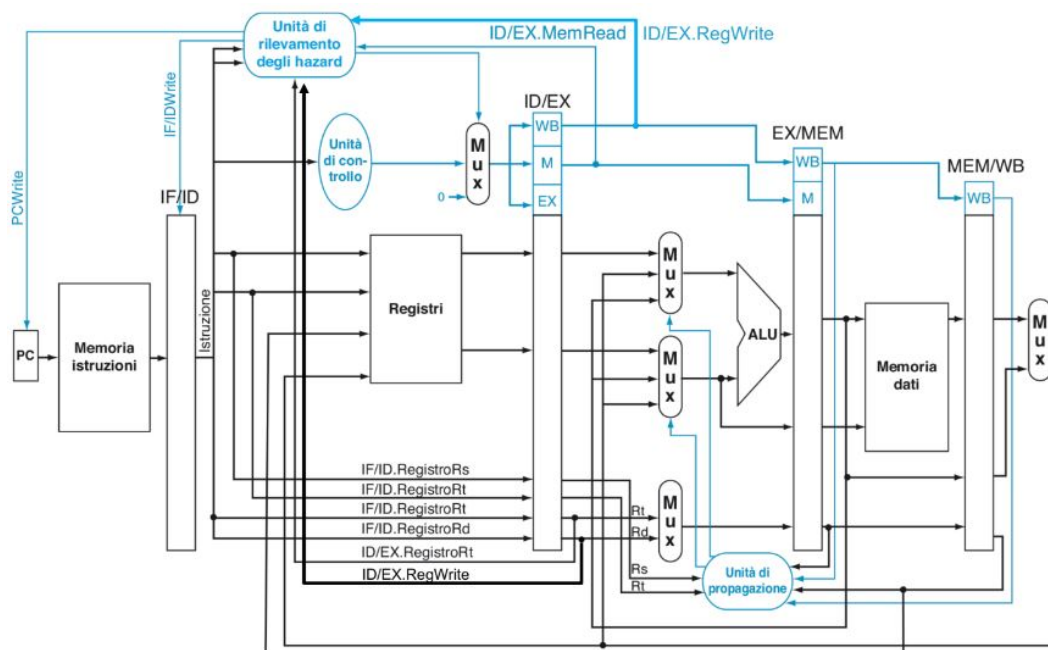


Figura 5.9: CPU con stallo

5.8 Anticipare i salti

Per quanto visto finora ogni istruzione viene riconosciuta nella fase ID mentre un'altra viene caricata, quindi il Jump implica almeno uno stallo per l'istruzione successiva. Inoltre, quando deve essere eseguito un salto, la pipeline è mezza piena e le istruzioni seguenti già caricate vanno eliminate dalla pipeline. Per eliminare l'istruzione con uno stallo, occorre (nella fase ID), annullare l'istruzione inutile (bolla), ovvero azzerare i segnali di controllo MemWrite e RegWrite e IF/ID, e aggiornare il PC affinché possa leggere la destinazione al prossimo colpo di clock.

Per saltare con la J in fase IF occorre: anticipare il riconoscimento della istruzione (che normalmente fa la CU in fase ID) comparando col valore dell'OpCode della J (000010); spostare la logica di aggiornamento del PC alla fase IF effettuando uno shift logico a 2 dei 26 bit meno significativi dell'istruzione e aggiungendo 4 bit più significativi di PC+4. Il risultato è una Jump anticipata che non introduce più stalli.

L'istruzione beq normalmente usa la ALU per fare il confronto, per cui: il salto avviene normalmente dopo la fase EXE (nella fase MEM); in caso di salto, le 2 istruzioni seguenti (già caricate) vanno annullate. Per anticipare la decisione di salto alla fase ID occorre non usare la ALU ma inserire un comparatore tra i due argomenti letti dal blocco registri, spostando la logica di salto ed il calcolo del salto relativo dalla fase EXE alla fase ID.

5.9 Predire i salti

Molte volte il responsabile di cali di efficienza è il tipo di codice che viene usato dalla beq, soprattutto nei cicli: se il test è all'inizio, seguito dal corpo del ciclo, la beq fa un solo salto alla fine del ciclo; se il test è alla fine, dopo il corpo, la beq effettua molti salti e in un solo caso continuerà.

La CPU vista finora presume che il salto non venga effettuato (branch not taken), se si potesse "prevedere" il salto si potrebbero caricare le istruzioni giuste ed evitare stalli.

Ad ogni istruzione di salto possiamo associare bit che contano i salti già effettuati per decidere se sia più probabile che il salto venga effettuato o meno. Con un bit si può rappresentare l'informazione l'ultima volta il salto è stato effettuato, ma nel realizzare un ciclo la previsione sarà sbagliata 2 volte: entrando e uscendo. Con due bit, invece, si può realizzare una macchina a stati finiti che necessita di due errori consecutivi per cambiare previsione, quindi incorre in meno errori nei cicli.

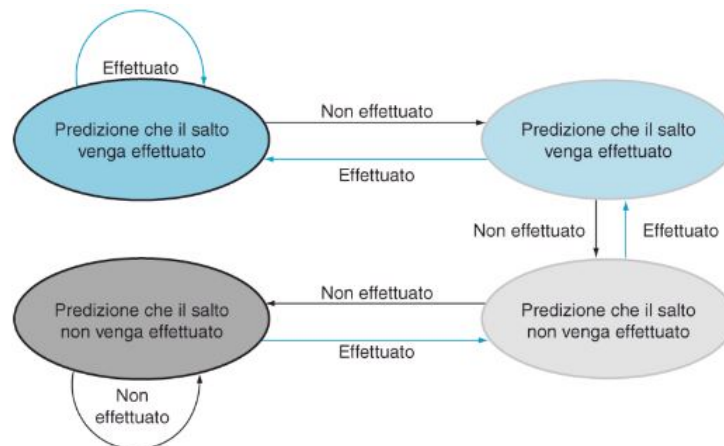


Figura 5.10: Macchina a stati finiti per la previsione dei salti

Capitolo 6

Cache

Fare tanti accessi alla memoria risulta essere un'operazione dispendiosa in termini di tempo (la RAM è circa 10-100 volte più lenta del processore) o di costi (una memoria tanto è veloce quanto costosa), per tanto vogliamo sviluppare un nuovo modello di memoria che ci aiuta in queste operazioni.

Quando utilizziamo la memoria non abbiamo bisogno di tutte le informazioni in essa contenute. Al pari della memoria umana, vorremmo dare priorità ai dati che vengono letti con maggiore frequenza e, di conseguenza, meno priorità a quelli che vengono letti di meno. Questa è l'idea alla base della *cache*, una memoria prioritaria, che si basa sul principio di località temporale: «un programma tende ad accedere allo stesso elemento in momenti vicini tra loro».

Possiamo vedere la cache come una memoria ridotta formata da N linee. Quando la CPU richiede un indirizzo, se il blocco corrispondente è presente nella cache allora il dato è caricato (*HIT*), altrimenti il dato è preso dalla RAM e il blocco è copiato nella cache (*MISS*).

Le N linee della cache contengono: 1 *bit di validità* che indica se la linea contiene dati validi, il campo *tag* che distingue quale blocco della memoria è nella linea evitando eventuali collisioni e il *blocco* memorizzato nella linea.

# linea	V	Tag	Blocco
0	0		
1	1	0	
2	1	1	
3	0		

Figura 6.1: Schema della cache

6.1 Direct mapping

Per guadagnare di efficienza la cache deve essere indipendente dal processore, per tanto le informazioni vengono memorizzate tramite il *direct mapping*. In questo modo non si deve interrogare la CPU ma basta semplicemente leggere l'indirizzo dell'istruzione per ottenere l'*offset*, l'*indice di linea* e il *tag*.

Numero di blocco																								Offset				
Tag																				Indice di linea				Word		Byte		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	1	0	1

Figura 6.2: Direct mapping

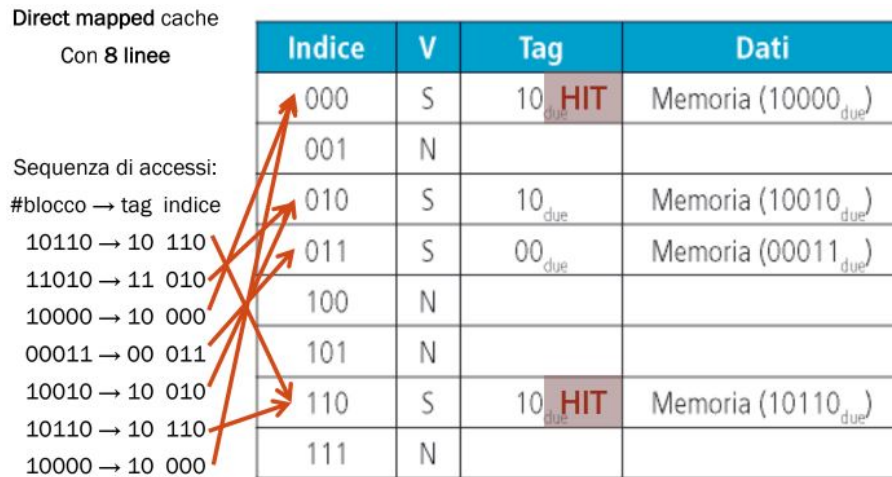


Figura 6.3: Esempio di direct mapping

Il blocco letto si posizionerà nella linea della cache ottenuta dal numero di blocco % (modulo) il numero di linee disponibili nella cache.

Esempio. Supponiamo di avere una cache che contiene 8 linee di cui ogni blocco può contenere 4 word = 16 byte, e l'indirizzo 2157 = 0x0000086D. Il numero di blocco si ottiene $2157/16 = 134 = 0x86$, a sua volta il numero di blocco si divide in tag e indice di linea: $\text{tag} = 134/8 = 16 = 0x10$; $\text{Indice di linea} = 134\%8 = 6 = 0x6$. Per l'offset infine calcoliamo $2157\%16 = 13 = 0xD$.

6.2 Determinare HIT/MISS

Supponiamo di avere una cache di 256 linee con blocchi da 16 word. Il procedimento per determinare se il processore trova che la posizione di memoria è in cache (*HIT*) o non è in memoria (*MISS*) è riassunto in Figura 6.4. Notiamo come in caso di *HIT*, si va all'indice di linea corrispondente, si verifica se il tag è uguale a quello del blocco e in tal caso il processore può leggere la parola direttamente dalla cache. In caso di *MISS*, invece, tramite un MUX, si prende dato relativo all'offset del blocco e si crea una nuova entità, che comprende l'etichetta appena richiesta dal processore e una copia del dato nella memoria principale.

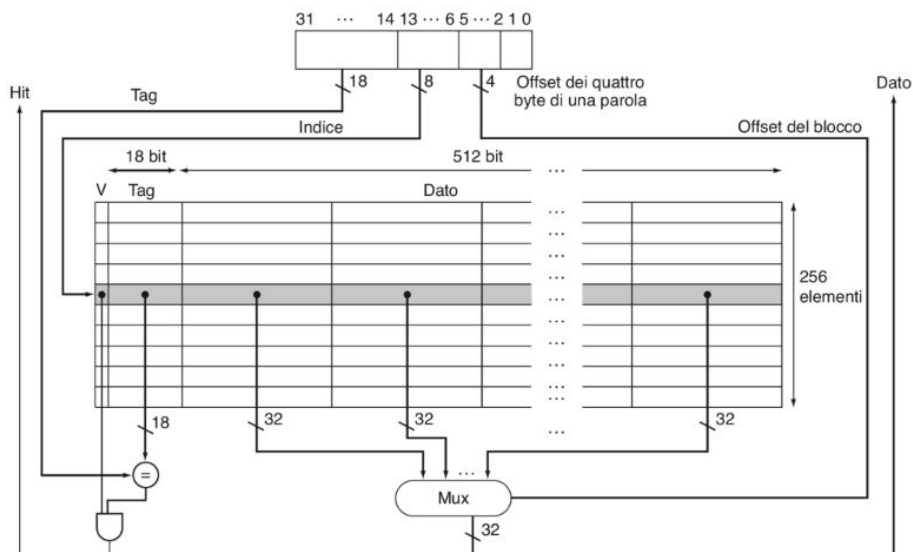


Figura 6.4: HIT e MISS

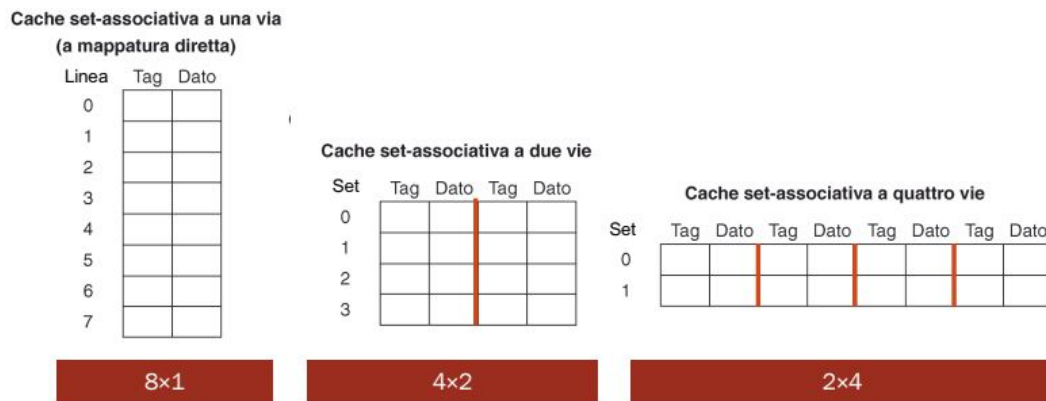


Figura 6.5: Tipi di set associative mapping

Un accesso alla cache può causare MISS per tre motivi:

1. *Cold start*: è la prima richiesta del dato/istruzione. Blocchi più grandi contengono più dati/istruzioni e il numero di cold miss diminuisce;
2. *Conflict*: il blocco è stato sovrascritto per via del grado di associatività della cache. Le cache con più vie hanno meno conflitti.
3. *Capacity*: il blocco è stato sovrascritto per via della capienza della cache. Con un maggior numero di linee si ottengono un maggior numero di richieste gestibili.

Abbiamo visto che in fase di MISS viene creata una nuova entità nella cache, che comprende l'etichetta richiesta dal processore e una copia del dato nella memoria principale. Ma come si aggiorna la memoria principale quando il dato modificato è in cache? Si può operare principalmente in due modi: tramite il *Write through* dove ad ogni modifica viene aggiornato il blocco in memoria o con il *Write back* dove il blocco viene aggiornato solo quando viene sostituito.

6.3 Altri tipi di mapping

La necessità di implementare altri tipi di mapping nasce dalla richiesta di un miglioramento nelle percentuali di HIT. Infatti con il Direct Mapping ogni blocco viene salvato in una linea fissata (che dipende dal numero di blocco), questo ci permette di avere un hardware più semplice e meno costoso (è facile determinare in quale linea cercare il dato), ma allo stesso tempo, blocchi diversi sono mappati sulla stessa linea, quindi se gli accessi a quei blocchi si alternano si verificano molti miss.

Un'altro tipo di mapping è il *Fully-associative mapping*, dove un blocco può essere posto in una linea qualsiasi. Questo garantisce massime prestazioni di flessibilità ma comporta la realizzazione di un hardware più complesso e più costoso.

La migliore soluzione è implementare il *Set-associative mapping*, dove le linee sono organizzate in S insiemi (set) e ogni blocco è disposto in una delle linee del set fissato.

A seconda del numero di blocchi possiamo avere vari tipi di set associative mapping. In Figura 6.5 ad esempio sono mostrati tre tipi di set mapping con 8 blocchi.

Possiamo ora rappresentare un'esempio di cache set-associativa a 4 vie con blocco da 1 word, vedi Figura 6.6

Per ogni via abbiamo una "tabella" che contiene S set (tanti quante sono le vie) con blocchi di N word. Ogni linea contiene 1 bit di validità (ed eventualmente i bit Used e Dirty) il tag da 32 bit composto da bit di offset e bit di indice di linea e il blocco da 8 [bit] × numero di byte.

Il grande vantaggio relativo all'utilizzo di un mapping di questo tipo riguarda l'attività dei dati. Infatti, implementando più vie per linea, i dati memorizzati saranno sovrascritti con meno frequenza in quanto saranno meno le possibilità di collisioni. Viene quindi naturale chiedersi: quando un set della cache è pieno quale blocco va sostituito? Si possono utilizzare diversi approcci ad esempio si può sostituire il blocco "più vecchio" LRU (*Least Recently Used*), oppure si può rimpiazzare il blocco "meno usato" LFU (*Least Frequently Used*) o ancora si può cambiare con un blocco a caso (*Random*).

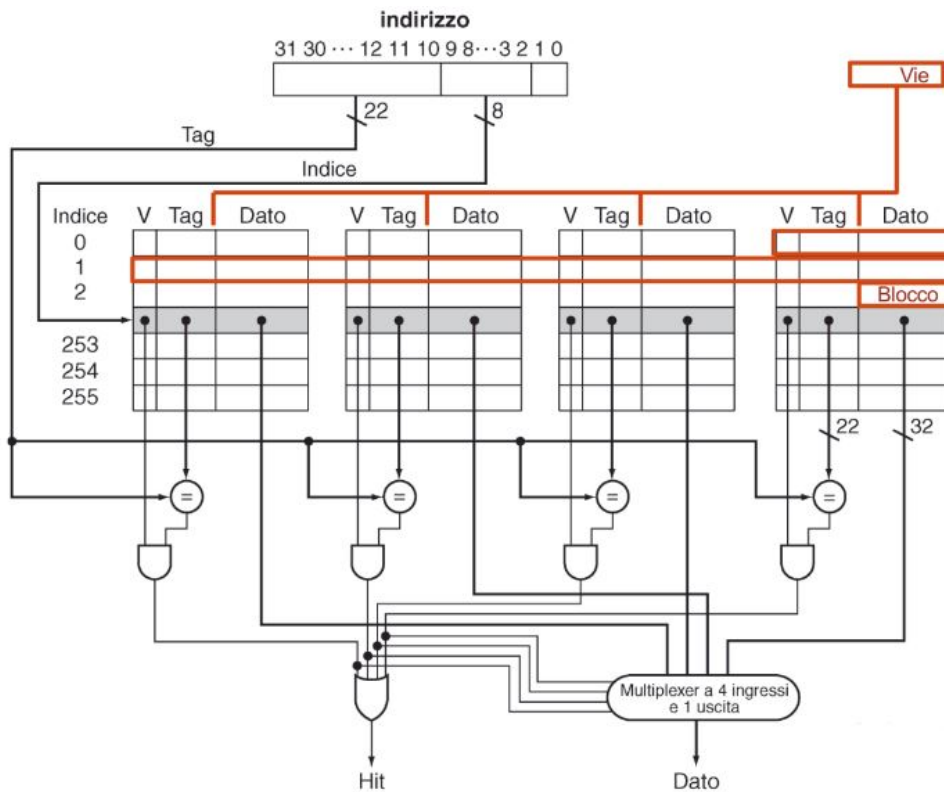


Figura 6.6: Cache set-associativa a 4 vie con blocco da 1 word

Modificare il blocco più vecchio richiederebbe di associare un bit (Used) a ciascuna linea e ad ogni accesso modificare Used = 1. Il valore sarà azzerato allo scadere di un intervallo di tempo così da avere le linee con Used = 1 più recenti mentre quelle con Used = 0 più vecchie. Un'implementazione di questo tipo risolve il problema iniziale ma è sicuramente costoso mantenere questa informazione. Per sostituire il blocco meno usato, invece, si può associare un contatore a ciascun set ed aggiornarlo ad ogni accesso. Anche in questo caso però abbiamo una limitazione, ovvero l'hardware risultante sarà necessariamente più complesso.

6.4 Cache multi-livello

Una cache multi-livello è formata da n cache impilate. In questo tipo di struttura è facile vedere come ogni HIT fornisce i dati a tutti i livelli superiori, mentre le MISS fanno richieste al livello subito inferiore.

In un sistema formato da cache multiple in parallelo, però, alcuni processi diversi potrebbero contemporaneamente accedere/modificare gli stessi dati. Per risolvere questo problema si possono collegare tra loro le cache in modo che ciascuna abbia nozione di cosa fanno le altre, e successivamente implementare la replicazione e migrazione di informazioni tra le cache. Per progettare un controllore di cache possiamo usare un *Automa a Stati Finiti (FSA)* (vedi Figura 6.7).

I 4 stati sono: attesa della richiesta (*idle*); ricerca del blocco in cache (*confronto dei tag*); lettura del blocco dalla memoria (*allocazione*); scrittura del blocco in memoria (*write-back*).

Utilizzare questo automa è un'ottimo punto di partenza, ma non risolve il problema iniziale. Infatti, più processori continuano a condividere la stessa memoria fisica (ad esempio, nei sistemi multi-core) e possono accedere contemporaneamente agli stessi dati tramite cache diverse (lo stesso blocco è presente in due copie diverse in due cache separate). Vogliamo quindi risolvere i problemi di coerenza e consistenza, dove per *coerenza* si indica quale valore deve essere restituito da una lettura, mentre per *consistenza* si indica quando, effettuata una scrittura, il dato sarà finalmente disponibile.

La soluzione definitiva consiste nell'implementare un *protocollo di snooping*: ogni cache che contiene un blocco ne ha anche lo stato di condivisione nelle altre cache; le cache sono collegate da un bus

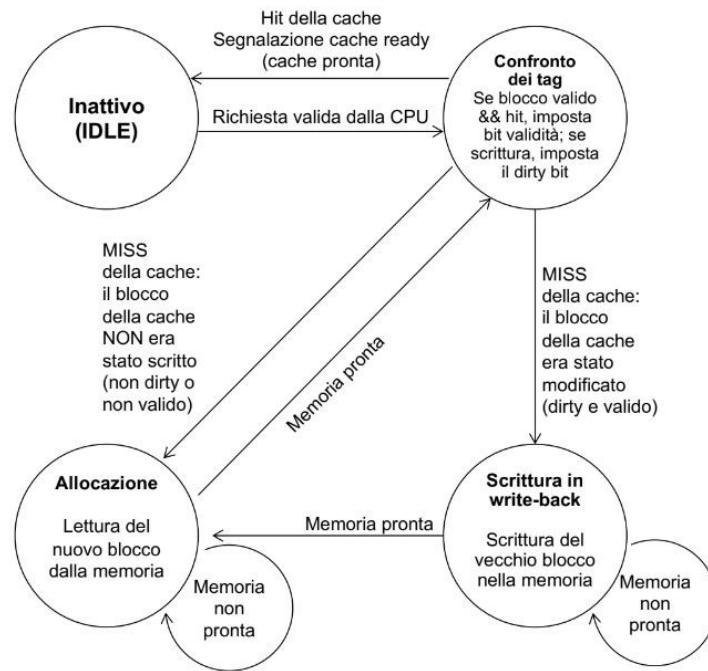


Figura 6.7: Automa a Stati Finiti della cache

o una rete che permette a ciascuna di sbirciare (snoop) le altre per aggiornare lo stato di condivisione del blocco; è sufficiente la trasmissione delle MISS e delle scritture da una cache a tutte le altre.

Capitolo 7

Memoria virtuale

In un sistema multi-processo la gestione della memoria è un argomento assai delicato. La memoria principale può non essere sufficiente per tutte le operazioni che devono essere effettuate, si vuole, in tal senso, rendere lo spazio di memoria per i processi indipendente dal limite della memoria disponibile.

Un'idea che viene utilizzata per ovviare a questo problema è:

1. produrre degli indirizzi "virtuali" (fittizi), che vengono successivamente trasformati in indirizzi "fisici" per accedere alla vera posizione dei dati;
2. suddividere la memoria in pagine, presenti in memoria fisica solo se necessario altrimenti memorizzate in una più lenta.

In questo modo lo spazio di indirizzamento di un processo si semplifica e gli spazi di memoria dei processi vengono separati e protetti da accessi esterni.

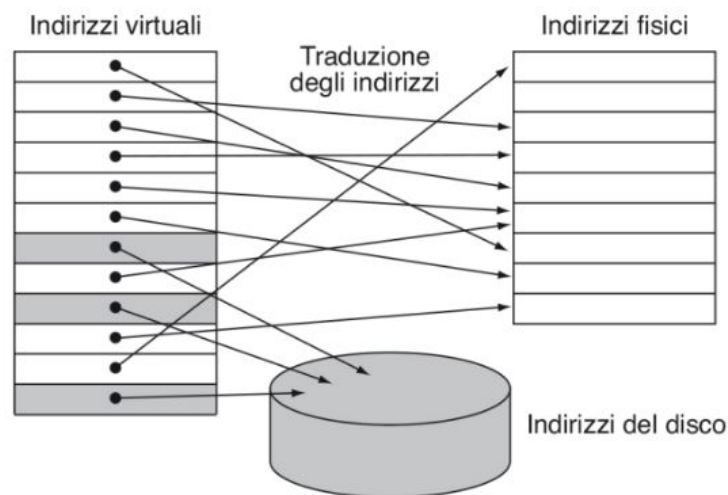


Figura 7.1: Memoria virtuale

7.1 Mapping degli indirizzi virtuali

L'indirizzo virtuale viene diviso in due campi il *numero di pagina* virtuale e l'*offset* (esattamente come per la cache). Questi dati vengono utilizzati per costruire una tabella delle pagine (*page table*), ossia una tabella contenente l'indirizzo delle pagine tramite cui si otterranno i relativi dati in memoria.

La page table contiene tre campi per ogni processo: *Valid*, *Dirty* e *Used* in base ai quali possiamo capire se l'indirizzo desiderato è presente o meno. In particolare se *Valid* è 0 vuol dire che si sta chiedendo per la prima volta una page, quindi la tabella indica l'indirizzo in memoria di massa e viene lanciata un'eccezione di *Page Fault* che richiede al sistema operativo di recuperare la pagina

e inserirla in memoria fisica. In tutti gli altri casi, la tabella indica l'indirizzo in memoria principale per ottenere il dato (sono necessari almeno 2 accessi: alla page table e alla memoria principale).

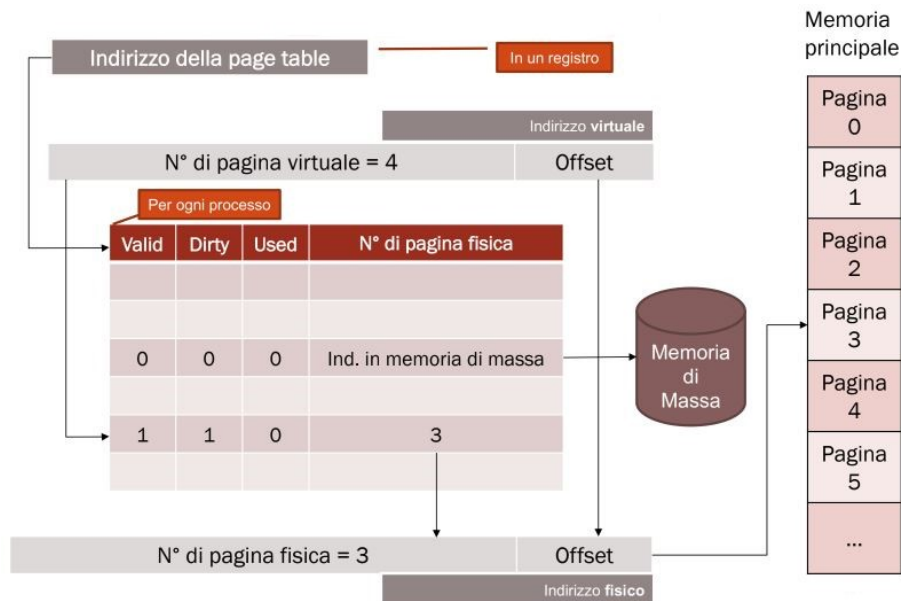


Figura 7.2: Page table

Come per la cache occorre sostituire pagine già usate e mantenere la coerenza dei dati. Rimpiazziamo le page tramite vari approcci:

- LRU (*Least Recently Used*) si sostituisce la pagina utilizzata meno di recente e si imposta il bit *Used* della page table a 1;
- LFU (*Least Frequently Used*) si sostituisce la pagina utilizzata meno di frequente (richiede un contatore delle frequenze);
- *Random* si sostituisce una pagina a caso.

e riscriviamo le page impostando il bit *Dirty* della page table a 1 per indicare che la pagina è stata modificata in memoria.

Per rendere più veloce l'accesso utilizza un *Table Lookaside Buffer* (TLB) che interagisce con le pagine più usate, può essere visto come una cache o meglio ancora come i preferiti del nostro browser. Gli indirizzi virtuali verranno dapprima cercati nel TLB, in caso di successo viene indicato l'indirizzo alla memoria fisica, altrimenti si passa il compito alla page table e si segue il metodo visto precedentemente (vedi Figura 7.4).

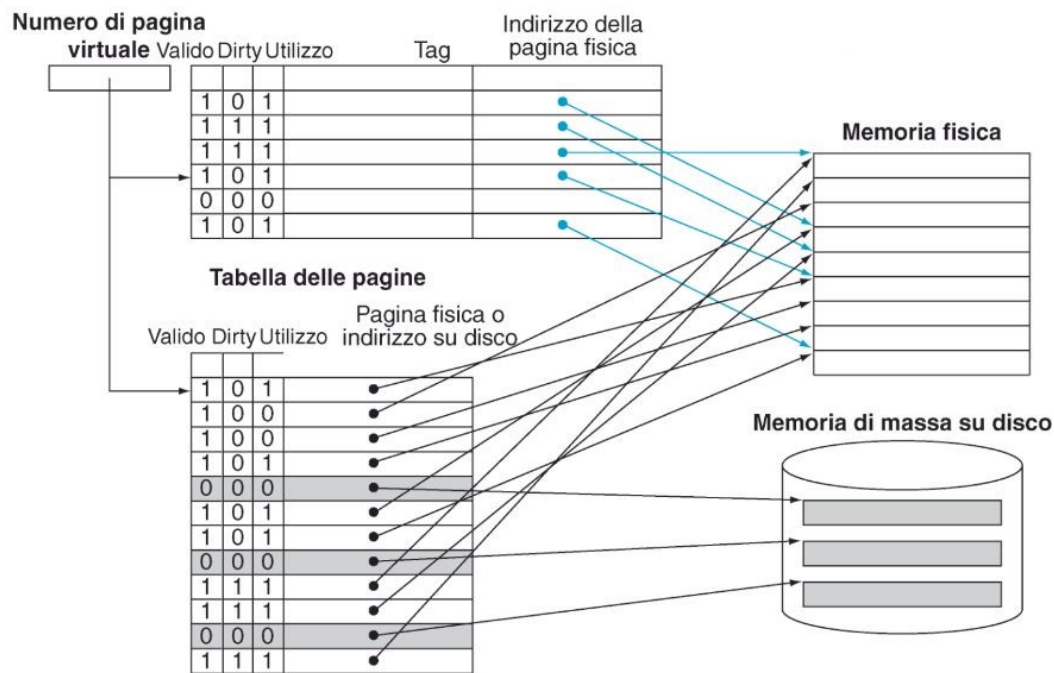


Figura 7.3: Table Lookaside Buffer

Infine l'accesso al dato (in memoria fisica) viene effettuato dalla cache. In genere la cache sul dato può essere indicizzata dall'indirizzo fisico (va svuotata al caricamento di una nuova pagina) oppure dall'indirizzo virtuale (va svuotata durante il context switch). Di seguito sono riportati i passaggi da seguire per entrambi gli approcci:

CPU $\xrightarrow{\text{virtuale}}$ Cache $\rightarrow$ TLB $\rightarrow$ Page Table $\xrightarrow{\text{fisico}}$ RAM

CPU $\xrightarrow{\text{virtuale}}$ TLB $\rightarrow$ Page Table $\xrightarrow{\text{fisico}}$ Cache $\rightarrow$ RAM

7.2 Supporto del Sistema Operativo

Il sistema operativo è l'unico gestore delle pagine fisiche e risponde alle eccezioni di Page Fault caricando in memoria fisica le pagine virtuali o modificandole sulle Page Table dei processi. In particolare:

- Assegna le nuove pagine fisiche ai processi che le richiedono;
- Recupera le pagine dai processi che hanno terminato l'esecuzione (o che le rilasciano);
- Impedisce che il processo possa accedere alle pagine di altri processi;
- Svuota il TLB per il process switch.

La CPU, inoltre, deve impedire che il programma possa fare operazioni "pericolose" pertanto gira in 2 modi: *user mode* per i programmi utente dove non sono disponibili le istruzioni per modificare Page Table e Process Table, Interruzioni ecc.; *kernel mode* solo per il Sistema Operativo dove sono disponibili tutte le istruzioni.

TLB	Tabella delle pagine	Cache	Possibile? Se sì, in quale situazione?
Hit	Hit	Miss	Possibile, sebbene la tabella delle pagine non venga mai realmente controllata se si verifica una hit del TLB.
Miss	Hit	Hit	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, il dato viene trovato nella cache.
Miss	Hit	Miss	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, si verifica però una miss della cache.
Miss	Miss	Miss	Miss del TLB, seguita da un page fault; al secondo tentativo, l'accesso al dato deve provocare una miss della cache.
Hit	Miss	Miss	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria.
Hit	Miss	Hit	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria.
Miss	Miss	Hit	Impossibile: i dati non possono trovarsi nella cache se la pagina non è presente in memoria.

Figura 7.4: Casistiche del mapping virtuale